

Random Neurons: ensemble de Perceptrons de Bolso com bootstrap e subespaços aleatórios

Ryan Alves

26 de abril de 2026

Enunciado

Implemente o “Random Neurons”, uma versão Random Forest muito malemolente. Considere a existência de um conjunto de treinamento $D = \{(x_i, y_i) \in \mathbb{R}^n \times \{1, 2, \dots, c\} : i = 1, \dots, m\}$ e siga o roteiro abaixo.

1 Item (a) – Bootstrap das amostras

Para um dado $L \in \mathbb{N}$ (parâmetro), obtenha L réplicas de D por meio de bootstrap. Denote cada uma dessas réplicas por D_l , com $l = 1, \dots, L$.

Para cada $l \in \{1, \dots, L\}$, foram sorteados m índices *com reposição* no intervalo $\{0, \dots, m-1\}$ por meio de `np.random.default_rng(...).integers(0, m, size=m)`. A réplica D_l é definida pelos pares correspondentes a esses índices. Em média, $\approx 63,2\%$ das amostras originais aparecem em cada D_l ; as demais são implicitamente *out-of-bag*. O código correspondente, já integrado dentro de `RandomNeurons.fit`, é:

```
1 idx = rng.integers(0, n, size=n)
2 Xb, yb = X[idx], y[idx]
```

2 Item (b) – Subespaços aleatórios de atributos

Sabendo que os padrões x_i em D são n -dimensionais, para cada réplica D_l , selecione aleatoriamente um número k entre 1 e n . Em seguida, selecione, sem repetição, k índices do conjunto $\{1, 2, \dots, n\}$. Armazene os índices sorteados em uma estrutura apropriada.

Para cada l , foi sorteado $k \sim \mathcal{U}\{1, \dots, n\}$ (uniforme discreta) e, em seguida, foram sorteados k índices *sem reposição* via `rng.choice(n, size=k, replace=False)`. O resultado é armazenado em uma lista `self.features_`, indexada pelo número do classificador. Essa estrutura é consultada novamente em tempo de predição (item (d)) para a seleção das colunas corretas de x .

```
1 k = int(rng.integers(1, d + 1))
2 feats = rng.choice(d, size=k, replace=False)
3 self.features_.append(feats)
```

3 Item (c) – Treinamento com Perceptron de Bolso e OVR

Para $l = 1, \dots, L$, utilize D_l , restrito aos atributos sorteados na etapa anterior, para treinar classificadores Perceptron de Bolso com número máximo de iterações igual a M . Para lidar com problemas multiclasse, adote a estratégia OVR.

Perceptron de Bolso (Pocket). É uma extensão do Perceptron clássico para casos não linearmente separáveis. Mantém em memória (“no bolso”) o vetor de pesos com o menor número de erros observado durante o treino. O viés é absorvido como uma coluna adicional de 1s em X . A cada iteração: (i) calcula as predições; (ii) sorteia uma amostra mal classificada; (iii) atualiza $w \leftarrow w + y_i x_i$; (iv) atualiza o “bolso” apenas se o novo w reduzir o erro empírico total.

One-vs-Rest (OVR). Para um problema com c classes, foram treinados c Pocket Perceptrons binários. Para a classe k , os rótulos são recodificados como $+1$ se $y_i = k$ e -1 caso contrário. A predição final escolhe a classe cujo classificador produz o maior valor de `decision_function`, isto é, $\hat{y} = \arg \max_k w_k^\top x$.

Integração com (a) e (b). Cada D_l é restrito às colunas `feats` sorteadas no item (b) *antes* do treinamento do OVRPocket com `max_iter = M`.

```
1 clf = OVRPocket(max_iter=self.M, random_state=seed)\
2     .fit(Xb[:, feats], yb)
3 self.models_.append(clf)
```

4 Item (d) – Votação por maioria

Para classificar um padrão não rotulado $x \in \mathbb{R}^n$, submeta-o a cada um dos L classificadores treinados (utilizando apenas os atributos exigidos por cada modelo) e tome como decisão final a classe de maior frequência entre as predições individuais (votação por maioria).

Para cada classificador l , são recuperados os atributos `featsl` armazenados no item (b) e utiliza-se apenas $x[\text{feats}_l]$ como entrada. Os L votos são coletados e a classe modal é devolvida:

```
1 votes = np.zeros((X.shape[0], self.L), dtype=int)
2 for j, (clf, feats) in enumerate(zip(self.models_, self.features_)):
3     votes[:, j] = clf.predict(X[:, feats])
4 out = np.empty(X.shape[0], dtype=int)
5 for i in range(X.shape[0]):
6     vals, counts = np.unique(votes[i], return_counts=True)
7     out[i] = vals[np.argmax(counts)]
8 return out
```

Em caso de empate, `np.argmax` devolve o menor índice, equivalente a um desempate determinístico pela menor classe.

5 Aplicação experimental

5.1 Datasets

- **Iris:** 150 amostras, 4 atributos, 3 classes balanceadas. Aproximadamente linearmente separável (*setosa* é trivial; *versicolor* e *virginica* têm leve sobreposição).
- **Wine:** 178 amostras, 13 atributos químicos, 3 classes levemente desbalanceadas. Atributos em escalas muito distintas; fronteiras predominantemente não lineares.

5.2 Protocolo

A divisão foi feita em 70/30 estratificada com `train_test_split`. Os atributos foram padronizados com `StandardScaler` ajustado *apenas no treino* (o Perceptron é sensível à escala). Cada configuração foi executada com 10 sementes diferentes; são reportadas a média e o desvio-padrão.

5.3 Grid de configurações

- $L \in \{5, 10, 25, 50, 100\}$ – número de classificadores no comitê;
- $M \in \{10, 50, 100, 500\}$ – máximo de iterações do Pocket.

5.4 Parâmetros do Random Forest

Para uma comparação justa, foi fixado `n_estimators` = L . Os demais parâmetros adotados são os *defaults* do scikit-learn, listados abaixo:

Parâmetro	Valor
<code>criterion</code>	'gini'
<code>max_depth</code>	None (cresce até folhas puras)
<code>min_samples_split</code>	2
<code>min_samples_leaf</code>	1
<code>max_features</code>	'sqrt' (heurística de Breiman)
<code>bootstrap</code>	True
<code>random_state</code>	fixo por execução

O Random Forest foi treinado sobre os dados *não padronizados*, pois árvores são invariantes a transformações monótonas em cada atributo.

6 Resultados

6.1 Random Neurons – efeito de L e M

A Figura 1 mostra a acurácia média da Random Neurons como função de L , para cada valor de M . A Figura 2 apresenta a mesma informação em forma de *heatmap*.

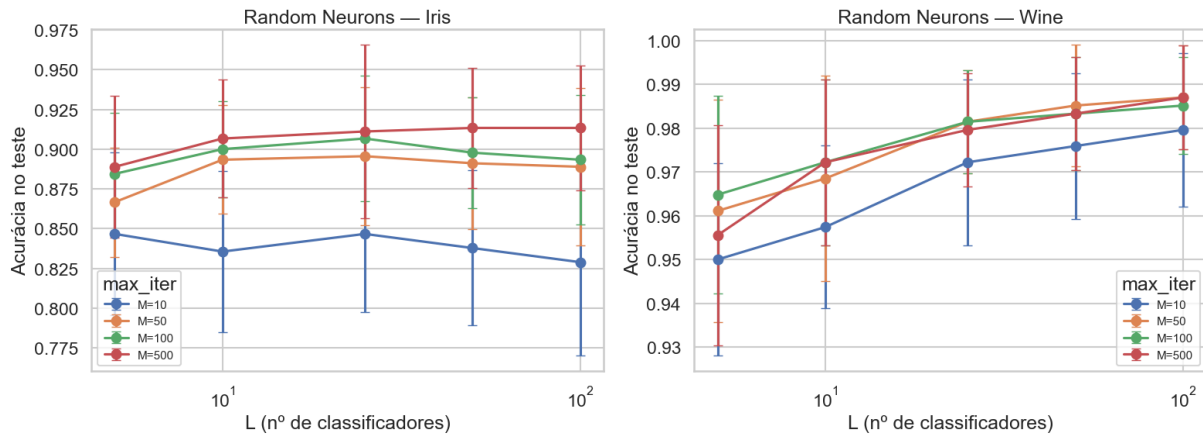


Figura 1: Acurácia média da Random Neurons em função de L , para cada M (Iris e Wine). Barras de erro: ± 1 desvio-padrão em 10 sementes.

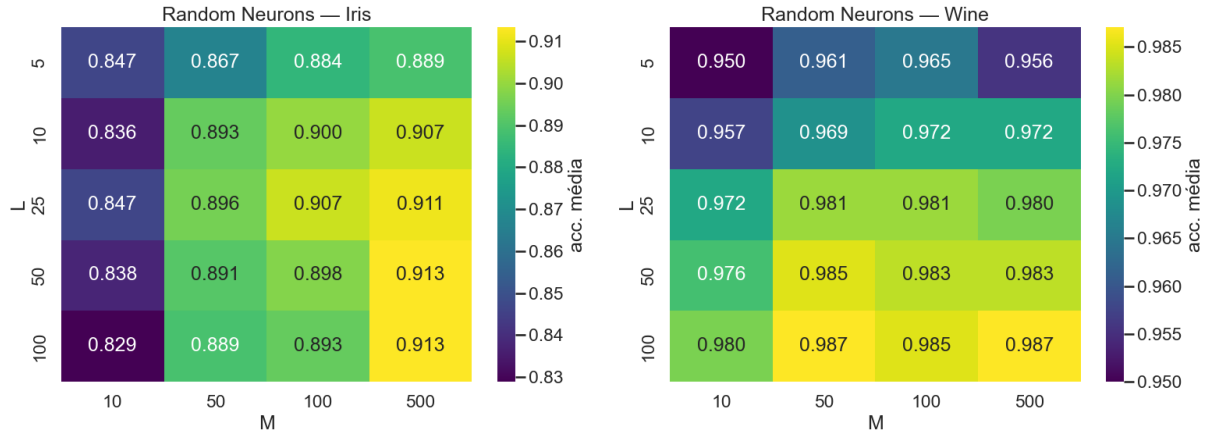


Figura 2: *Heatmaps* da acurácia média da Random Neurons no grid $L \times M$.

6.2 Random Neurons vs. Random Forest

A Figura 3 compara os dois métodos para o mesmo L , com $M = 100$ fixado na Random Neurons.

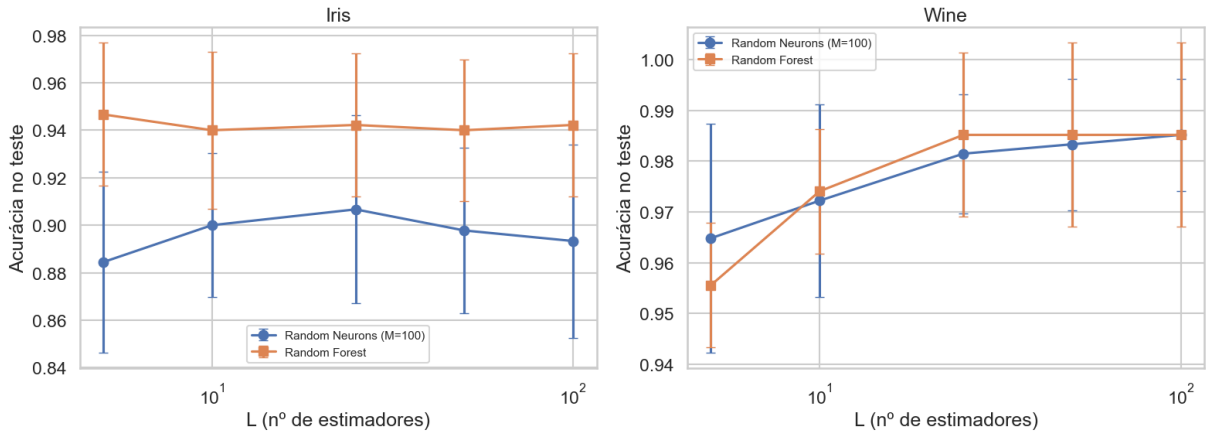


Figura 3: Random Neurons ($M = 100$) vs. Random Forest, mesmo L .

6.3 Tabela de acurácia

A Tabela 1 resume as melhores configurações encontradas em cada dataset.

Tabela 1: Melhores configurações encontradas em cada dataset.

Dataset	Modelo	Configuração	Acurácia média	Desvio
Iris	Random Neurons	$L = 50, M = 500$	0.913	0.038
Iris	Random Forest	$L = 5$	0.947	0.030
Wine	Random Neurons	$L = 100, M = 50$	0.987	0.012
Wine	Random Forest	$L \geq 25$	0.985	0.016

7 Discussão crítica

Efeito de L e M . O aumento de L reduz a variância da predição em ambos os datasets, com saturação a partir de $L \approx 25-50$ – comportamento típico de *bagging*. Quanto a M , o valor

$M = 10$ é claramente insuficiente (até ≈ 6 pontos abaixo no Iris); já $M \in \{50, 100, 500\}$ são estatisticamente equivalentes, indicando que o “bolso” converge cedo.

Random Neurons vs. Random Forest. O resultado depende do dataset. No Iris ($n = 4$), o RF é superior por ≈ 3 pontos percentuais (0.947 vs. 0.913): a baixa dimensionalidade torna a hipótese “linear sobre $k \leq 4$ atributos” muito restritiva. No Wine ($n = 13$), os métodos são estatisticamente equivalentes, com leve vantagem da Random Neurons para $L \geq 50$ (0.987 vs. 0.985): a alta dimensionalidade favorece o ensemble linear, pois há mais subespaços onde as classes ficam aproximadamente separáveis por um hiperplano.

8 Conclusão

A Random Neurons funciona como esperado: *bagging* com subespaços aleatórios estabiliza um classificador-base intrinsecamente fraco (Pocket Perceptron) e o leva a desempenho competitivo. Em datasets de baixa dimensionalidade (Iris) o Random Forest mantém vantagem; em dimensionalidade moderada (Wine) os dois métodos são estatisticamente equivalentes.